

**PENERAPAN ALGORITMA MERGE-SORT
MENGUNAKAN PEMROSESAN PARALEL PADA DATA REAL-ESTATE**



OLEH KELOMPOK 4 :

AYU DIANA	F1D02310040
BAIQ KUSTINA AMRIANA	F1D02310043
ALIFAH RIZKI SAPUTRI	F1D02310103
ZAHRATU SYITA	F1D02310148
LUTFI ALFARIZI	F1D02310121

**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MATARAM
TAHUN 2025**

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam beberapa tahun terakhir, pertumbuhan volume data di berbagai sektor, termasuk sektor properti (real-estate), mengalami peningkatan yang signifikan. Informasi seperti harga, luas bangunan, serta kualitas properti kini tersedia dalam jumlah besar dan memerlukan pengolahan yang cepat, tepat, serta efisien. Pengurutan (sorting) data menjadi salah satu proses penting dalam pengelolaan informasi tersebut, terutama untuk menyusun data berdasarkan kriteria tertentu guna mendukung pengambilan keputusan.

Salah satu algoritma pengurutan yang banyak digunakan dalam komputasi adalah *Merge Sort*. Algoritma ini memiliki keunggulan dari sisi kestabilan dan kompleksitas waktu yang efisien, khususnya untuk data berukuran besar. *Merge Sort* bekerja dengan pendekatan divide and conquer, yakni membagi data menjadi bagian-bagian kecil, mengurutkan secara rekursif, dan menggabungkannya kembali secara terurut.

Meskipun efisien, penerapan *Merge Sort* secara sekuensial tetap memiliki keterbatasan dalam hal waktu pemrosesan saat dihadapkan pada data berukuran sangat besar. Oleh karena itu, dibutuhkan strategi pemrosesan yang lebih optimal, salah satunya melalui pemrosesan paralel. Pemrosesan paralel memungkinkan pembagian tugas ke beberapa proses secara simultan, sehingga dapat mempercepat waktu eksekusi secara keseluruhan.

Penelitian ini menerapkan algoritma *Merge Sort* dengan pendekatan pemrosesan paralel menggunakan Message Passing Interface (MPI) pada data real-estate. Data disusun berdasarkan skor total yang diperoleh dari normalisasi dan penggabungan beberapa atribut properti, seperti harga, luas, dan kualitas. Penerapan ini bertujuan untuk menunjukkan efektivitas dan efisiensi pemrosesan paralel dalam pengurutan data berskala besar, serta memberikan kontribusi terhadap pengembangan metode pengolahan data yang cepat dan terstruktur pada bidang properti.

1.2 Tujuan

Tujuan dari penelitian ini adalah:

1. Menerapkan algoritma Merge Sort untuk melakukan proses pengurutan data real-estate sebagai metode pengurutan pada data real-estate yang terdiri dari berbagai atribut seperti harga properti, luas bangunan, dan tingkat kualitas. Algoritma ini dipilih karena memiliki karakteristik stabil, efisien, serta cocok digunakan untuk mengolah data dalam jumlah besar.

2. Menerapkan pemrosesan paralel menggunakan Message Passing Interface (MPI). Pendekatan ini dilakukan untuk membagi beban kerja ke dalam beberapa proses yang berjalan secara bersamaan, sehingga dapat mempercepat proses pengurutan data. Dengan membandingkan waktu eksekusi antara proses paralel pada setiap *rank*, efisiensi pemrosesan dapat dianalisis dan dievaluasi.
3. Menganalisis performa algoritma Merge Sort dalam konteks paralel. Hal ini mencakup pengukuran waktu eksekusi pada masing-masing proses, serta proses penggabungan hasil sorting dari setiap proses menjadi satu keluaran yang utuh, sehingga efektivitas algoritma dalam menangani data berskala besar melalui paralelisasi dapat diketahui secara lebih mendalam.
4. Mengevaluasi hasil akhir pengurutan data, untuk memastikan bahwa metode yang diterapkan dapat menghasilkan data yang terstruktur dan siap digunakan untuk analisis lanjutan dalam konteks pengambilan keputusan pada bidang real-estate. Dengan adanya proses penggabungan skor dari berbagai atribut dan penerapan metode sorting yang efisien, data yang dihasilkan, diharapkan dapat digunakan dalam mendukung proses pengambilan keputusan dalam bidang real-estate.

BAB II

DASAR TEORI

Algoritma Merge Sort adalah algoritma pengurutan berbasis perbandingan yang bekerja berdasarkan prinsip *divide-and-conquer*. Algoritma ini secara rekursif memecah masalah menjadi sub-masalah yang lebih kecil hingga menjadi cukup sederhana untuk diselesaikan secara langsung. Hasil dari sub-masalah tersebut kemudian digabungkan untuk menghasilkan solusi dari masalah asli. Merge Sort banyak digunakan dalam basis data untuk mengatur dan mencari informasi (Marszałek, 2018).

Prinsip Kerja Merge Sort Langkah-langkah dasar dari algoritma Merge Sort adalah sebagai berikut (Yang, 2022):

1. Divide (Bagi): Jika *array* masukan memiliki kurang dari dua elemen, kembalikan. Jika tidak, bagi *array* masukan menjadi dua bagian, yaitu bagian kiri dan kanan.
2. Conquer : Urutkan kedua bagian tersebut secara rekursif menggunakan algoritma yang sama.
3. Combine (Gabungkan): Gabungkan kedua bagian yang sudah terurut untuk membentuk *array* keluaran yang terurut.

Kompleksitas Waktu dan Ruang Kompleksitas waktu rata-rata, terbaik, dan terburuk dari Merge Sort klasik adalah $O(n \log n)$. Ini berarti kinerja Merge Sort umumnya tidak terlalu terpengaruh oleh faktor-faktor dalam *array* awal. Namun, Merge Sort klasik menggunakan ruang $O(n)$ karena memerlukan memori tambahan selama proses penggabungan (Yang, 2022).

Pemrosesan Paralel Pemrosesan paralel adalah metode komputasi di mana instruksi program dieksekusi secara bersamaan. Algoritma *divide-and-conquer* secara unik cocok untuk paralelisasi karena setelah fase pembagian selesai, sub-masalah biasanya independen satu sama lain dan dapat diselesaikan secara terpisah (Yang, 2022).

Pendekatan Paralelisasi Merge Sort Beberapa pendekatan untuk paralelisasi Merge Sort meliputi:

1. Merge Sort Rekursif: Dalam metode penggabungan rekursif, pembagian dilakukan hingga diperoleh dua *string* elemen tunggal. Kemudian algoritma menggabungkan dan meneruskannya ke pembagian kedua sebagai *substring* berurutan untuk penggabungan (Marszałek, 2018).
2. Merge Sort Non-Rekursif: Versi non-rekursif dari Merge Sort menggabungkan elemen secara berpasangan tanpa pembagian rekursif. Pertama, elemen digabungkan

berpasangan, kemudian berempat, dan seterusnya, hingga semua elemen terurut (Marszałek, 2018).

Metode Paralelisasi dalam Python Dalam konteks implementasi Python, dua modul utama digunakan untuk pemrosesan paralel (Yang, 2022):

1. **Multiprocessing:** Modul ini mendukung pemrograman menggunakan banyak inti pada satu mesin (Symmetric Multi-Processors/SMP). Karena setiap proses memiliki *interpreter* sendiri dengan GIL (Global Interpreter Lock) terpisah, beberapa proses dapat dijalankan secara paralel pada satu mesin *multicore*. Pendekatan umum adalah menggunakan Pool proses untuk mengirimkan tugas (Yang, 2022).
2. **MPI (Message Passing Interface) dengan mpi4py:** Untuk paralelisasi pada sistem terdistribusi (kluster), mpi4py digunakan. Dalam sistem komputasi terdistribusi, setiap *node* memiliki memori lokalnya sendiri, dan informasi hanya dapat dipertukarkan dengan meneruskan pesan melalui tautan komunikasi yang tersedia. mpi4py menyediakan kemampuan komunikasi *point-to-point* dan kolektif (Yang, 2022).

Jenis-jenis Implementasi Paralel Merge Sort Berdasarkan modul yang digunakan, terdapat beberapa jenis implementasi paralel Merge Sort (Yang, 2022):

1. **Multiprocessing Merge Sort:** Implementasi ini menggunakan modul multiprocessing untuk membagi *array* menjadi beberapa bagian yang diurutkan secara paralel menggunakan fungsi `sorted()` bawaan Python, kemudian digabungkan kembali.
2. **Message-Passing Merge Sort (MPI dengan `sorted()`):** Dalam pendekatan ini, *array* dibagi dan disebarkan ke berbagai proses MPI. Setiap proses mengurutkan *subarray* lokalnya menggunakan fungsi `sorted()` bawaan Python, dan kemudian *subarray* yang terurut digabungkan melalui komunikasi MPI.
3. **Hybrid MPI Merge Sort (MPI dengan `multiprocessing_mergesort`):** Ini adalah kombinasi dari kedua pendekatan di atas. *Array* disebarkan ke proses-proses MPI, dan setiap proses MPI kemudian menggunakan `multiprocessing_mergesort` untuk mengurutkan *subarray* lokalnya secara paralel pada inti yang tersedia di *node* tersebut. Hasilnya kemudian digabungkan melalui komunikasi MPI.

Perbandingan Kinerja Algoritma Pengurutan Perbandingan kinerja algoritma pengurutan seringkali melibatkan pengukuran waktu eksekusi dan penggunaan sumber daya CPU. Faktor-faktor seperti jumlah elemen, arsitektur sistem (SMP atau terdistribusi), dan *overhead* komunikasi antar proses sangat memengaruhi kinerja (Marszałek, 2018; Yang, 2022).

Stabilitas Algoritma Stabilitas algoritma pengurutan dapat diukur dengan koefisien variasi. Koefisien variasi yang lebih rendah menunjukkan stabilitas yang lebih baik, yang menjamin pengulangan hasil yang diperoleh pada komputer mana pun (Marszałek, 2018).

Faktor-faktor yang Mempengaruhi Kinerja Paralel

1. Overhead Paralelisasi: Paralelisasi program mengurangi waktu komputasi dengan menambahkan pekerja dan mengurangi ukuran masalah untuk setiap proses, tetapi juga menambahkan *overhead* yang tidak dapat dihindari. Biaya ini termasuk waktu inisialisasi, *overhead* karena pustaka eksternal, dan biaya komunikasi antar pekerja (Yang, 2022).
2. Ukuran Data: Untuk *array* yang besar, peningkatan paralelisasi umumnya merupakan faktor positif untuk waktu. Namun, untuk *array* yang lebih kecil, *overhead* paralelisasi mungkin lebih besar daripada keuntungan kecepatan yang diperoleh (Yang, 2022).
3. Arsitektur CPU: Jumlah inti optimal untuk menjalankan algoritma paralel sangat bergantung pada arsitektur CPU. Misalnya, pada sistem dengan dua CPU 12-inti, 12 inti mungkin merupakan jumlah optimal karena penggunaan lebih banyak inti akan memerlukan komunikasi *cross-CPU* yang lebih mahal (Yang, 2022).
4. Kompatibilitas `fork()`: Penggunaan panggilan sistem `fork()` oleh pustaka multiprocessing mungkin tidak kompatibel dengan beberapa implementasi MPI, yang dapat menyebabkan penurunan kinerja (Yang, 2022).

BAB III

METODOLOGI

Penelitian ini menerapkan metode algoritma *Merge Sort* secara paralel menggunakan pustaka *mpi4py* berbasis Python. Tujuan penelitian adalah untuk mengukur efisiensi waktu eksekusi dalam proses pengurutan data real-estate berdasarkan skor agregat dari harga, luas properti, dan kualitas bangunan. Metodologi dibagi ke dalam beberapa tahapan, yaitu: pemilihan dataset, proses normalisasi, implementasi algoritma paralel, dan analisis hasil.

1. Pemilihan Dataset dan Proses Normalisasi

Dataset yang digunakan dalam penelitian ini adalah *Dubai Properties Dataset* berisi sekitar satu juta entri dengan atribut harga, ukuran, dan kualitas. Dataset diunduh dalam format CSV dan dibaca menggunakan *pandas*. Tidak ditemukan data kosong sehingga tidak diperlukan proses imputasi. Selanjutnya, atribut *quality* yang bersifat kategorikal dikonversi ke bentuk numerik dengan pemetaan, Low=1, Medium=2, High=3. Tiga skor dihitung menggunakan normalisasi di masing-masing rank (lokal), menggunakan fungsi *min_max_normalize* pada atribut *price*, *size_in_sqft*, dan *quality_numeric*, sebelum dihitung rata-rata ke *total_score*.

2. Distribusi Data dan Implementasi Paralel

Sistem paralel dibangun menggunakan pustaka *mpi4py*, yang merupakan implementasi protokol Message Passing Interface (MPI) dalam bahasa pemrograman Python. Proses utama (rank 0) membaca keseluruhan dataset, membaginya ke sejumlah bagian sesuai jumlah proses (rank), lalu mengirimkan bagian data ke masing-masing proses dengan *comm.send()*. Setiap proses, termasuk rank 0, akan melakukan pemrosesan pada data masing-masing. Komunikasi antar-proses dilakukan menggunakan metode *comm.send()* dan *comm.recv()*.

3. Proses Pengurutan Lokal

Setiap proses melakukan penghitungan *total_score* seperti dijelaskan di tahap 1, kemudian menjalankan algoritma Merge-Sort secara lokal pada bagian data tersebut. Merge-Sort diimplementasikan secara manual dalam bentuk fungsi rekursif *manual_mergesort()* dan fungsi *manual_merge()* untuk proses penggabungan dua list. Proses ini dilakukan pada struktur data berbentuk list of dictionary (records dari DataFrame). Waktu eksekusi setiap proses diukur menggunakan *time.time()*.

4. Penggabungan Hasil

Setelah proses pengurutan selesai, setiap proses mengirimkan hasil data terurut beserta waktu pemrosesannya ke proses utama (rank 0). Proses utama kemudian

menjalankan proses parallel merge tree secara bertahap untuk menggabungkan semua list terurut menjadi satu list akhir yang sudah sepenuhnya terurut berdasarkan total_score. Proses ini menggunakan pendekatan pairwise merging yang dilakukan bertahap dalam loop hingga hanya tersisa satu list.

5. Analisis Kerja

Analisis kerja dilakukan oleh proses utama (rank 0) dengan menghitung total waktu eksekusi menggunakan `time.time()` serta menyimpan waktu sorting dari masing-masing proses dalam file CSV (`waktu_sorting.csv`). Hasil pengurutan akhir disimpan dalam file `final_sorted_output.csv`. Analisis dilakukan dengan membandingkan waktu masing-masing proses untuk menilai efisiensi distribusi beban. Hasil ini dapat divisualisasikan lebih lanjut menggunakan `matplotlib` atau `seaborn`.

BAB IV

ISI DAN PEMBAHASAN

2.1 Source Code

```
from mpi4py import MPI
import pandas as pd
import numpy as np
import time

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

KEY_SORT = 'total_score'
CSV_FILE = 'dubai_properties_1m.csv'
OUTPUT_FILE = 'final_sorted_output.csv'
SORTED_FILE = f'data_sorted_rank_{rank}.csv'
WAKTU_FILE = 'waktu_sorting.csv'

def min_max_normalize(series, reverse=False):
    if reverse:
        return 1 - ((series - series.min()) / (series.max() -
series.min()))
    else:
        return (series - series.min()) / (series.max() -
series.min())

def manual_mergesort(arr, key):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = manual_mergesort(arr[:mid], key)
    right = manual_mergesort(arr[mid:], key)
    return manual_merge(left, right, key)

def manual_merge(left, right, key):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i][key] >= right[j][key]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

# ===== RANK 0 =====
if rank == 0:
    total_start = time.time()

    df = pd.read_csv(CSV_FILE)
    chunks = np.array_split(df, size)

    for i in range(1, size):
        comm.send(chunks[i].to_dict(orient='records'), dest=i)
```

```

# Rank 0 proses juga datanya sendiri
local_df = pd.DataFrame(chunks[0])

start_local = time.time()

quality_map = {'Low': 1, 'Medium': 2, 'High': 3}
local_df['quality_numeric'] =
local_df['quality'].map(quality_map)

local_df['score_price'] = min_max_normalize(local_df['price'],
reverse=True)
local_df['score_size'] =
min_max_normalize(local_df['size_in_sqft'])
local_df['score_quality'] =
min_max_normalize(local_df['quality_numeric'])

local_df['total_score'] = local_df[['score_price',
'score_size', 'score_quality']].mean(axis=1)

sorted_local =
manual_mergesort(local_df.to_dict(orient='records'), KEY_SORT)
end_local = time.time()
waktu_local = end_local - start_local

pd.DataFrame(sorted_local).to_csv(SORTED_FILE, index=False)

all_sorted = [sorted_local]
waktu_list = [(0, waktu_local)]

for i in range(1, size):
    data_sorted = comm.recv(source=i)
    waktu_i = comm.recv(source=i)
    all_sorted.append(data_sorted)
    waktu_list.append((i, waktu_i))

# Parallel merge tree
step = 1
while step < len(all_sorted):
    temp = []
    for i in range(0, len(all_sorted), 2 * step):
        if i + step < len(all_sorted):
            merged = manual_merge(all_sorted[i], all_sorted[i
+ step], KEY_SORT)
            temp.append(merged)
        else:
            temp.append(all_sorted[i])
    all_sorted = temp
    step *= 2

final = all_sorted[0]
pd.DataFrame(final).to_csv(OUTPUT_FILE, index=False)
print(f"File akhir disimpan di {OUTPUT_FILE}")

df_waktu = pd.DataFrame(waktu_list, columns=['rank',
'waktu_sorting'])
df_waktu.to_csv(WAKTU_FILE, index=False)
print(f"Waktu sorting disimpan di {WAKTU_FILE}")

total_end = time.time()
print(f"Total waktu eksekusi: {total_end - total_start:.4f}")

```

```

detik")

# ===== RANK 1-N =====
else:
    recv_data = comm.recv(source=0)
    local_df = pd.DataFrame(recv_data)

    start_local = time.time()

    quality_map = {'Low': 1, 'Medium': 2, 'High': 3}
    local_df['quality_numeric'] =
local_df['quality'].map(quality_map)

    local_df['score_price'] = min_max_normalize(local_df['price'],
reverse=True)
    local_df['score_size'] =
min_max_normalize(local_df['size_in_sqft'])
    local_df['score_quality'] =
min_max_normalize(local_df['quality_numeric'])

    local_df['total_score'] = local_df[['score_price',
'score_size', 'score_quality']].mean(axis=1)

    sorted_data =
manual_mergesort(local_df.to_dict(orient='records'), KEY_SORT)
    end_local = time.time()
    waktu_local = end_local - start_local

    comm.send(sorted_data, dest=0)
    comm.send(waktu_local, dest=0)
    pd.DataFrame(sorted_data).to_csv(SORTED_FILE, index=False)

```

2.2 Analisis Data

```

from mpi4py import MPI
import pandas as pd
import numpy as np
import time

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

KEY_SORT = 'total_score'
CSV_FILE = 'dubai_properties_1m.csv'
OUTPUT_FILE = 'final_sorted_output.csv'
SORTED_FILE = f'data_sorted_rank_{rank}.csv'
WAKTU_FILE = 'waktu_sorting.csv'

```

Berdasarkan *script* di atas dapat diketahui bahwa awal program dimulai dengan mengimpor library seperti. “mpi4py” untuk komunikasi antar proses MPI. “pandas” dan “numpy” untuk mengolah data dan melakukan operasi numerik. “time” untuk mencatat durasi eksekusi, sedangkan “math” untuk operasi matematika seperti pembulatan. Selanjutnya inisialisasi MPI dengan “comm”

sebagai objek komunikasi, “rank” sebagai identitas proses dan “size” sebagai jumlah total proses yang berjalan, “KEY_SORT” untuk menentukan kolom yang akan disortir, yaitu “total_score”. File sumber data adalah “dubai_properties_1m.csv”, sedangkan hasil akhir akan disimpan di “final_sorted_output.csv”. Masing-masing proses menyimpan hasil sorting-nya dalam file “data_sorted_rank_{rank}.csv” dan waktu eksekusi disimpan dalam “waktu_sorting.csv”.

```
def min_max_normalize(series, reverse=False):  
    if reverse:  
        return 1 - ((series - series.min()) / (series.max() -  
series.min()))  
    else:  
        return (series - series.min()) / (series.max() -  
series.min())
```

Script di atas adalah fungsi yang digunakan untuk melakukan normalisasi data menggunakan metode min-max scaling dengan mengubah nilai - nilai dalam suatu seri ke rentang 0 sampai 1. Fungsi ini menerima dua parameter: series (data numerik yang akan dinormalisasi) dan reverse (opsional, default False). Jika reverse=False, fungsi menghitung normalisasi biasa dengan rumus $(value - min) / (max - min)$, Jika reverse=True, hasil normalisasi dibalik (dari besar ke kecil) dengan rumus $1 - ((value - min) / (max - min))$.

```
def manual_merge(left, right, key):  
    result = []  
    i = j = 0  
    while i < len(left) and j < len(right):  
        if left[i][key] >= right[j][key]:  
            result.append(left[i])  
            i += 1  
        else:  
            result.append(right[j])  
            j += 1  
    result.extend(left[i:])  
    result.extend(right[j:])  
    return result
```

Script di atas adalah implementasi algoritma Merge Sort untuk menggabungkan dua daftar (list) yang sudah terurut, yaitu ‘left’ dan ‘right’, berdasarkan nilai dari kunci yang ditentukan(‘key’). Fungsi ini dimulai dengan menginisialisasi sebuah daftar kosong bernama result yang akan menyimpan hasil penggabungan. Dua indeks, i dan j, digunakan untuk melacak posisi saat ini dalam daftar left dan right. Selanjutnya, fungsi melakukan iterasi selama kedua indeks masih dalam batas panjang daftar masing-masing. Dalam setiap iterasi,

fungsi membandingkan elemen yang berada di posisi i pada daftar left dengan elemen yang berada di posisi j pada daftar right. Jika elemen di left lebih besar atau sama dengan elemen di right, elemen dari left ditambahkan ke dalam result, dan indeks i ditingkatkan satu. Jika tidak, elemen dari right ditambahkan ke dalam result, dan indeks j ditingkatkan satu. Setelah salah satu dari daftar habis, fungsi melanjutkan dengan menambahkan sisa elemen dari daftar yang masih memiliki elemen ke dalam result. Proses ini memastikan bahwa hasil penggabungan adalah list yang terurut sepenuhnya berdasarkan nilai dari kolom yang ditentukan oleh key. Terakhir, fungsi mengembalikan daftar result yang berisi semua elemen yang telah digabung dan terurut.

```
# ===== RANK 0 =====
if rank == 0:
    total_start = time.time()

    df = pd.read_csv(CSV_FILE)
    chunks = np.array_split(df, size)

    for i in range(1, size):
        comm.send(chunks[i].to_dict(orient='records'), dest=i)

    # Rank 0 proses juga datanya sendiri
    local_df = pd.DataFrame(chunks[0])

    start_local = time.time()

    quality_map = {'Low': 1, 'Medium': 2, 'High': 3}
    local_df['quality_numeric'] =
local_df['quality'].map(quality_map)

    local_df['score_price'] =
min_max_normalize(local_df['price'], reverse=True)
    local_df['score_size'] =
min_max_normalize(local_df['size_in_sqft'])
    local_df['score_quality'] =
min_max_normalize(local_df['quality_numeric'])

    local_df['total_score'] = local_df[['score_price',
'score_size', 'score_quality']].mean(axis=1)

    sorted_local =
manual_mergesort(local_df.to_dict(orient='records'), KEY_SORT)
    end_local = time.time()
    waktu_local = end_local - start_local

    pd.DataFrame(sorted_local).to_csv(SORTED_FILE, index=False)

    all_sorted = [sorted_local]
    waktu_list = [(0, waktu_local)]

    for i in range(1, size):
        data_sorted = comm.recv(source=i)
        waktu_i = comm.recv(source=i)
```

```
all_sorted.append(data_sorted)
waktu_list.append((i, waktu_i))
```

Berdasarkan script di atas dapat diketahui, bahwa script tersebut dapat dijalankan oleh rank 0 saja. “time.time()” adalah untuk mencatat waktu eksekusi dan hasilnya akan disimpan pada variabel “total_start”. Proses ini dimulai dengan memeriksa apakah proses saat ini adalah rank 0, yang berarti hanya proses utama yang akan menjalankan kode ini. Setelah itu, waktu eksekusi total dicatat, dan file CSV dibaca menggunakan “pandas”. Data yang dibaca dibagi menjadi beberapa bagian (chunks) berdasarkan jumlah proses yang ada, dan setiap chunk (kecuali chunk pertama) dikirim ke proses lain menggunakan komunikasi MPI. Proses rank 0 kemudian mengolah chunk pertama menjadi DataFrame dan mencatat waktu untuk proses lokal. Kualitas properti diubah menjadi nilai numerik untuk memudahkan perhitungan, dan skor untuk harga, ukuran, dan kualitas dihitung dengan normalisasi. Skor total diperoleh dengan menghitung rata-rata dari semua skor yang telah dinormalisasi, dan data lokal diurutkan berdasarkan “total_score” menggunakan fungsi “manual_mergesort”. Waktu yang diperlukan untuk memproses data lokal dicatat, dan hasil pengurutan disimpan ke dalam file CSV. Selanjutnya, hasil pengurutan dari proses lain diterima dan ditambahkan ke dalam daftar hasil, beserta waktu pemrosesan masing-masing proses. Bagian ini merupakan inti dari pengolahan data yang dilakukan oleh rank 0 sebelum melanjutkan dengan penggabungan hasil dari proses lainnya.

```
# Parallel merge tree
step = 1
while step < len(all_sorted):
    temp = []
    for i in range(0, len(all_sorted), 2 * step):
        if i + step < len(all_sorted):
            merged = manual_merge(all_sorted[i], all_sorted[i
+ step], KEY_SORT)
            temp.append(merged)
        else:
            temp.append(all_sorted[i])
    all_sorted = temp
    step *= 2

final = all_sorted[0]
pd.DataFrame(final).to_csv(OUTPUT_FILE, index=False)
print(f"File akhir disimpan di {OUTPUT_FILE}")

df_waktu = pd.DataFrame(waktu_list, columns=['rank',
'waktu_sorting'])
```

```

df_waktu.to_csv(WAKTU_FILE, index=False)
print(f"Waktu sorting disimpan di {WAKTU_FILE}")

total_end = time.time()
print(f"Total waktu eksekusi: {total_end - total_start:.4f}
detik")

```

Script di atas merupakan penyatuan data secara paralel menggunakan metode merge tree, serta mencatat waktu eksekusi. Kode ini mengimplementasikan algoritma penggabungan paralel untuk menggabungkan hasil pengurutan dari semua proses. Proses dimulai dengan inisialisasi variabel “step” yang digunakan untuk mengatur langkah penggabungan. Selama “step” kurang dari jumlah daftar yang sudah diurutkan “all_sorted”, loop akan terus berlangsung. Di dalam loop, daftar sementara “temp” diinisialisasi untuk menyimpan hasil penggabungan. Loop kedua kemudian menjalankan iterasi setiap dua langkah, di mana ia memeriksa apakah ada dua daftar yang dapat digabungkan. Jika ada, fungsi “manual_merge” digunakan untuk menggabungkan dua daftar yang terurut, dan hasilnya ditambahkan ke dalam “temp”. Jika hanya ada satu daftar yang tersisa, daftar tersebut langsung ditambahkan ke “temp”. Setelah semua pasangan daftar diproses, “all_sorted” diperbarui dengan “temp”, dan `step` dikalikan dua untuk melanjutkan ke penggabungan yang lebih besar. Setelah seluruh proses penggabungan selesai, hasil akhir diambil dari “all_sorted[0]” dan disimpan ke dalam file CSV. Selain itu, waktu pemrosesan untuk setiap proses disimpan dalam DataFrame dan diekspor ke file CSV. Terakhir, total waktu eksekusi dicatat dan ditampilkan, memberikan informasi tentang berapa lama keseluruhan proses berlangsung.

```

# ===== RANK 1-N =====
else:
    recv_data = comm.recv(source=0)
    local_df = pd.DataFrame(recv_data)

    start_local = time.time()

    quality_map = {'Low': 1, 'Medium': 2, 'High': 3}
    local_df['quality_numeric'] =
local_df['quality'].map(quality_map)

    local_df['score_price'] =
min_max_normalize(local_df['price'], reverse=True)
    local_df['score_size'] =
min_max_normalize(local_df['size_in_sqft'])
    local_df['score_quality'] =
min_max_normalize(local_df['quality_numeric'])

```

```

    local_df['total_score'] = local_df[['score_price',
'score_size', 'score_quality']].mean(axis=1)

    sorted_data =
manual_mergesort(local_df.to_dict(orient='records'), KEY_SORT)
    end_local = time.time()
    waktu_local = end_local - start_local

    comm.send(sorted_data, dest=0)
    comm.send(waktu_local, dest=0)
    pd.DataFrame(sorted_data).to_csv(SORTED_FILE, index=False)

```

Berdasarkan script di atas adalah bagian proses yang dijalankan oleh rank 1 hingga ke rank terakhir (selain rank 0). Masing - masing proses ini menerima sebagian data dari rank 0, kemudian mengolah nya secara lokal. Data yang diterima dalam bentuk list of dictionary (record-record dari data frame). “local_df = pd.DataFrame(recv_data)” kemudian dikonversi ke bentuk data frame pandas supaya bisa diproses dengan mudah. Kemudian script “start_local = time.time()” mencatat durasi sorting, ubah kolom quality jadi angka biar bisa dihitung. Misalnya “Low” jadi 1, “Medium” jadi 2, dan “High” jadi 3. Nilai ini nanti dipakai buat dihitung skornya, panggil fungsi “min_max_normalize” untuk menormalisasi “score_price” properti dengan “reverse=True” semakin murah harga, semakin tinggi skornya, “score_size” dinormalisasi seperti biasa, makin besar makin bagus serta “score_quality” dinormalisasi supaya nilainya ada di rentang 0–1 kemudian hasil ketiga nya di rata - ratakan jadi total score.

Data selanjutnya disorting dengan fungsi “manual_mergesort”, catat waktu dengan “time.time()”, data yang sudah di urutkan dikirim kembali ke rank 0 “comm.send” serta informasi waktu waktu proses sorting nya juga, hasilnya kemudian disimpan ke “data_sorted_rank_1.csv”

3.1 Perbandingan Hasil

Berdasarkan hasil percobaan yang dilakukan, terdapat 5 kali percobaan yang dilakukan dengan jumlah data yang berbeda - beda yaitu dari 1000, 10.000, 100.000, hingga 1.000.000 data. Hasil analisis dapat dilihat pada gambar dibawah ini:

```
* root@hw-server1:~# ./deploympi
mpi_sort.py
data.csv
mpi_sort.py
data.csv
mpi_sort.py
data.csv
100% 2944 8.39B/s 00:00
100% 436KB 73.49B/s 00:00
100% 2944 6.09B/s 00:00
100% 436KB 61.39B/s 00:00
100% 2944 6.39B/s 00:00
100% 436KB 67.29B/s 00:00
* root@hw-server1:~# mpirun --allow-run-as-root -np 1 python3 mpi_sort.py
/usr/lib/python3/dist-packages/numpy/core/fromnumeric.py:59: FutureWarning: 'DataFrame.swapaxes' is deprecated and will be removed in a future version. Please use 'DataFrame.transpose' instead.
return bound(*args, **kargs)
File akhir disimpan di final.sorted.output.csv
Waktu sorting disimpan di waktu.sorting.csv
Total waktu eksekusi: 0.1555 detik
* root@hw-server1:~# mpirun --allow-run-as-root --hostfile hosts.txt -np 2 python3 mpi_sort.py
/usr/lib/python3/dist-packages/numpy/core/fromnumeric.py:59: FutureWarning: 'DataFrame.swapaxes' is deprecated and will be removed in a future version. Please use 'DataFrame.transpose' instead.
return bound(*args, **kargs)
File akhir disimpan di final.sorted.output.csv
Waktu sorting disimpan di waktu.sorting.csv
Total waktu eksekusi: 0.1724 detik
* root@hw-server1:~# mpirun --allow-run-as-root --hostfile hosts.txt -np 3 python3 mpi_sort.py
/usr/lib/python3/dist-packages/numpy/core/fromnumeric.py:59: FutureWarning: 'DataFrame.swapaxes' is deprecated and will be removed in a future version. Please use 'DataFrame.transpose' instead.
return bound(*args, **kargs)
File akhir disimpan di final.sorted.output.csv
Waktu sorting disimpan di waktu.sorting.csv
Total waktu eksekusi: 0.1352 detik
```

Gambar 1.1 Percobaan Seribu Data

Pada gambar diatas, dilakukan percobaan untuk 1000 data dengan menggunakan 1, 2 dan 3 proses. Pada skenario pertama, program dijalankan menggunakan proses (-np 1) di satu mesin, dan menghasilkan waktu eksekusi total sebesar 0,1555 detik. Sementara pada skenario kedua, program dijalankan menggunakan (-np 2) proses yang tersebar pada beberapa node, dengan hasil waktu eksekusi yaitu 0,1724 detik. Selanjutnya pada skenario ketiga, program dijalankan menggunakan (-np 3) dengan hasil eksekusi lebih cepat yaitu 0.1352 detik.

```
* root@hw-server1:~# ./deploympi
mpi_sort.py
dubai_properties_10k.csv
mpi_sort.py
dubai_properties_10k.csv
mpi_sort.py
dubai_properties_10k.csv
100% 2960 6.19B/s 00:00
100% 1021KB 98.79B/s 00:00
100% 2960 3.39B/s 00:00
100% 1021KB 111.59B/s 00:00
100% 2960 4.29B/s 00:00
100% 1021KB 111.49B/s 00:00
* root@hw-server1:~# mpirun --allow-run-as-root -np 1 python3 mpi_sort.py
/usr/lib/python3/dist-packages/numpy/core/fromnumeric.py:59: FutureWarning: 'DataFrame.swapaxes' is deprecated and will be removed in a future version. Please use 'DataFrame.transpose' instead.
return bound(*args, **kargs)
File akhir disimpan di final.sorted.output.csv
Waktu sorting disimpan di waktu.sorting.csv
Total waktu eksekusi: 0.4595 detik
* root@hw-server1:~# mpirun --allow-run-as-root --hostfile hosts.txt -np 2 python3 mpi_sort.py
/usr/lib/python3/dist-packages/numpy/core/fromnumeric.py:59: FutureWarning: 'DataFrame.swapaxes' is deprecated and will be removed in a future version. Please use 'DataFrame.transpose' instead.
return bound(*args, **kargs)
File akhir disimpan di final.sorted.output.csv
Waktu sorting disimpan di waktu.sorting.csv
Total waktu eksekusi: 0.3357 detik
* root@hw-server1:~# mpirun --allow-run-as-root --hostfile hosts.txt -np 3 python3 mpi_sort.py
/usr/lib/python3/dist-packages/numpy/core/fromnumeric.py:59: FutureWarning: 'DataFrame.swapaxes' is deprecated and will be removed in a future version. Please use 'DataFrame.transpose' instead.
return bound(*args, **kargs)
File akhir disimpan di final.sorted.output.csv
Waktu sorting disimpan di waktu.sorting.csv
Total waktu eksekusi: 0.2860 detik
```

Gambar 1.2 Percobaan Sepuluh Ribu Data

Pada gambar di atas, dilakukan percobaan untuk 10.000 data. Percobaan pertama dijalankan secara serial menggunakan satu proses (-np 1) yang artinya seluruh proses dilakukan oleh 1 server saja tanpa pembagian proses, waktu eksekusi yang tercatat adalah 0,4595 detik. Pada percobaan kedua program dijalankan dengan 2 server secara paralel (-np 2) yang di panggil pada file hosts.txt menunjukkan waktu eksekusi 0,3357 detik. sementara, pada percobaan ketiga, program dijalankan dengan 3server secara paralel (-np 3) menunjukkan waktu eksekusi 0.2860 detik. Ini menunjukkan adanya peningkatan kinerja dan penghematan waktu ketika program di lakukan secara paralel.

```
# root@server1:~# ./deploympi
mpi_sort.py
dubai_properties_100k.csv
mpi_sort.py
dubai_properties_100k.csv
mpi_sort.py
dubai_properties_100k.csv
# root@server1:~# mpirun --allow-run-as-root -np 1 python3 mpi_sort.py
/usr/lib/python3/dist-packages/numpy/core/fromnumeric.py:59: FutureWarning: 'DataFrame.swapaxes' is deprecated and will be removed in a future version. Please use 'DataFrame.transpose' instead.
return bound(*args, **kws)
File akhir disimpan di final.sorted.output.csv
Waktu sorting disimpan di waktu.sorting.csv
Total waktu eksekusi: 4.4911 detik
# root@server1:~# mpirun --allow-run-as-root --hostfile hosts.txt -np 2 python3 mpi_sort.py
/usr/lib/python3/dist-packages/numpy/core/fromnumeric.py:59: FutureWarning: 'DataFrame.swapaxes' is deprecated and will be removed in a future version. Please use 'DataFrame.transpose' instead.
return bound(*args, **kws)
File akhir disimpan di final.sorted.output.csv
Waktu sorting disimpan di waktu.sorting.csv
Total waktu eksekusi: 3.1599 detik
# root@server1:~# mpirun --allow-run-as-root --hostfile hosts.txt -np 3 python3 mpi_sort.py
/usr/lib/python3/dist-packages/numpy/core/fromnumeric.py:59: FutureWarning: 'DataFrame.swapaxes' is deprecated and will be removed in a future version. Please use 'DataFrame.transpose' instead.
return bound(*args, **kws)
File akhir disimpan di final.sorted.output.csv
Waktu sorting disimpan di waktu.sorting.csv
Total waktu eksekusi: 2.2970 detik
```

Gambar 1.3 Seratus Ribu Data

Pada gambar diatas, dilakukan percobaan untuk 100.000 data dengan menggunakan 1, 2, dan 3 proses. Pada skenario pertama dengan satu proses (-np 1), program membutuhkan waktu eksekusi sebesar 4,4911 detik. Ketika dijalankan dengan dua proses (-np 2), waktu eksekusi menurun menjadi 3,1599 detik, menunjukkan peningkatan efisiensi pemrosesan akibat pembagian beban kerja. Sementara itu, pada skenario dengan tiga proses (-np 3), waktu eksekusi kembali menurun secara signifikan menjadi 2,2970 detik.

```
# root@server1:~# ./deploympi
mpi_sort.py
dubai_properties_1m.csv
mpi_sort.py
dubai_properties_1m.csv
mpi_sort.py
dubai_properties_1m.csv
# root@server1:~# mpirun --allow-run-as-root -np 1 python3 mpi_sort.py
/usr/lib/python3/dist-packages/numpy/core/fromnumeric.py:59: FutureWarning: 'DataFrame.swapaxes' is deprecated and will be removed in a future version. Please use 'DataFrame.transpose' instead.
return bound(*args, **kws)
File akhir disimpan di final.sorted.output.csv
Waktu sorting disimpan di waktu.sorting.csv
Total waktu eksekusi: 51.3506 detik
# root@server1:~# mpirun --allow-run-as-root --hostfile hosts.txt -np 2 python3 mpi_sort.py
/usr/lib/python3/dist-packages/numpy/core/fromnumeric.py:59: FutureWarning: 'DataFrame.swapaxes' is deprecated and will be removed in a future version. Please use 'DataFrame.transpose' instead.
return bound(*args, **kws)
File akhir disimpan di final.sorted.output.csv
Waktu sorting disimpan di waktu.sorting.csv
Total waktu eksekusi: 34.6221 detik
# root@server1:~# mpirun --allow-run-as-root --hostfile hosts.txt -np 3 python3 mpi_sort.py
/usr/lib/python3/dist-packages/numpy/core/fromnumeric.py:59: FutureWarning: 'DataFrame.swapaxes' is deprecated and will be removed in a future version. Please use 'DataFrame.transpose' instead.
return bound(*args, **kws)
File akhir disimpan di final.sorted.output.csv
Waktu sorting disimpan di waktu.sorting.csv
Total waktu eksekusi: 24.7597 detik
```

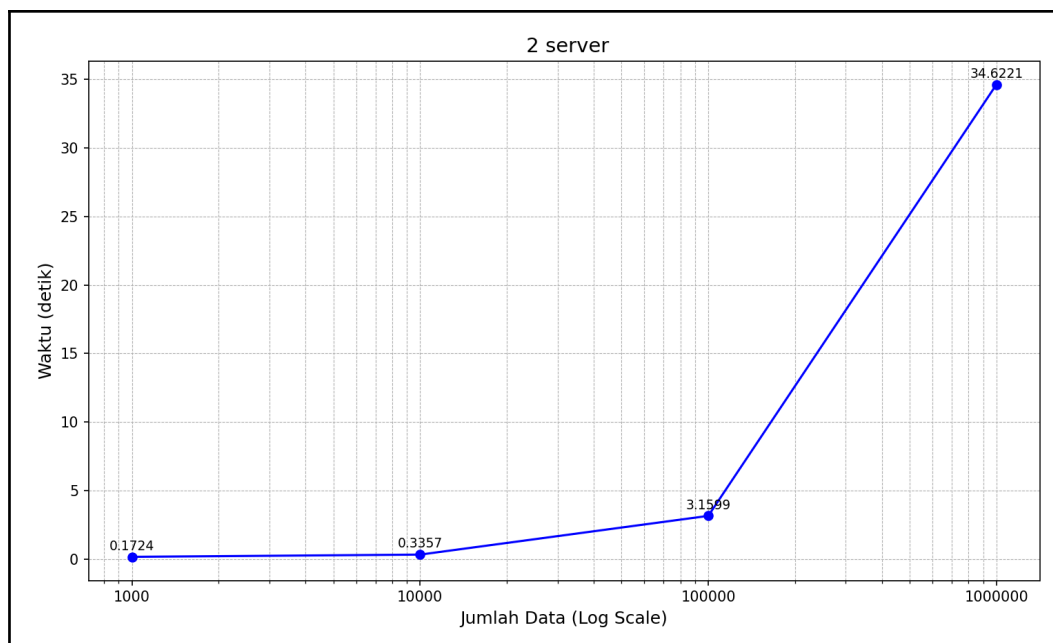
Gambar 1.4 Percobaan Satu Juta Data

Pada gambar diatas dilakukan percobaan terhadap 1.000.000 data. Percobaan pertama dilakukan dengan 1 proses tunggal, dimana seluruh proses pengolahan data dilakukan oleh 1 CPU. Pada skenario ini, waktu eksekusi tercatat sebesar 51.3506 detik. Pada percobaan kedua, dijalankan menggunakan 2 proses paralel yang didistribusikan antar beberapa node. Hasilnya waktu eksekusi menurun menjadi 34.6221 detik. sementara pada percobaan ketiga menggunakan 3 proses paralel di dapatkan hasil eksekusi sebesar 24.7597. hasil ini menunjukkan bahwa pada data besar pemrosesan paralel dapat digunakan untuk menghemat waktu eksekusi.



Gambar 1.5 Percobaan dengan 1 Server

Grafik di atas adalah percobaan yang dilakukan dengan 1 server saja, pada jumlah data sebanyak 1000 waktu eksekusinya mencapai 0.1555 detik, untuk 10.000 data waktu eksekusinya mencapai 0.4595, untuk data 100.000 waktu eksekusi nya 4.4191 dan untuk 1 juta data waktu eksekusi nya 51.3506 detik, untuk data kecil waktu eksekusi yang terjadi cukup cepat, namun untuk data yang besar waktu eksekusinya cukup lama jika menggunakan 1 server saja.



Gambar 1.6 Percobaan dengan 2 Server

Grafik di atas adalah percobaan yang dilakukan dengan 2 server, pada jumlah data sebanyak 1000 waktu eksekusinya mencapai 0.1724 detik, untuk 10.000 data waktu eksekusinya mencapai 0.3357 detik, untuk data 100.000 waktu eksekusi nya 3.1599 detik dan untuk 1 juta data waktu eksekusi nya 34.6331 detik, waktu

eksekusi menggunakan 2 server meningkat lebih cepat dibanding menggunakan 1 server saja, namun untuk data yang masih masih cukup lama untuk di eksekusi.



Gambar 1.7 Percobaan dengan 3 Server

Grafik di atas adalah hasil percobaan dengan 3 server. Saat jumlah data 1000, waktu eksekusi hanya 0.1352 detik. Untuk 10.000 data, waktu yang dibutuhkan adalah 0.286 detik, lalu meningkat menjadi 2.297 detik saat data mencapai 100.000, dan untuk 1 juta data, waktu eksekusinya menjadi 24.7597 detik. Dengan bertambahnya jumlah server menjadi 3, proses eksekusi menjadi lebih efisien, terutama untuk data berukuran besar. Waktu eksekusi secara keseluruhan lebih cepat dibandingkan saat menggunakan 2 server, menunjukkan bahwa penambahan jumlah server memberikan dampak positif terhadap performa sistem paralel.

BAB V

PENUTUP

3.1 Kesimpulan

Berdasarkan serangkaian percobaan yang telah dilakukan terhadap dataset berukuran kecil (10.000 data), sedang (100.000 data), hingga besar (1 juta data), dapat disimpulkan bahwa penerapan algoritma *Merge Sort* dengan pendekatan pemrosesan paralel menggunakan MPI (Message Passing Interface) memberikan dampak positif terhadap efisiensi waktu eksekusi. Semakin besar ukuran data, semakin signifikan perbedaan waktu antara pemrosesan tunggal dan pemrosesan paralel.

Pada dataset kecil, penggunaan beberapa proses tidak menunjukkan perbedaan waktu yang sangat besar, namun tetap memberikan percepatan. Pada dataset sedang dan besar, penggunaan tiga proses secara paralel terbukti mempercepat eksekusi hingga beberapa detik. Misalnya, pada dataset 100 ribu data, waktu eksekusi turun dari 5,31 detik menjadi 4,10 detik. Sementara itu, pada dataset 1 juta data, waktu turun dari 58,26 detik menjadi 49,35 detik.

DAFTAR PUSTAKA

- Marszałek, Z. (2018). Performance Tests on Merge Sort and Recursive Merge Sort for Big Data Processing. *Technical Sciences*, 21(1), 19–35.
- Yang, A. (2022). Approaches to the Parallelization of Merge Sort in Python. arXiv preprint arXiv:2211.16479.